Codez
@0xCodez

Loop engineering: the 14-step roadmap from prompter to loop designer.

Most developers still prompt their coding agents by hand. They type, they wait, they read the diff, they type again. **9**out of 10 builders have never written a single loop that prompts the agent for them.

No **automation**, no **state file**, no **verifier**, no **schedule**. The leverage point has moved - from *typing prompts* to *designing systems that prompt*. This is the 14-step roadmap from prompter to loop designer.

> Follow my Linkedin to get fresh AI alpha: linkedin.com/in/lev-deviatkin

This is the 14-step roadmap to make that shift - sourced from Anthropic's engineering docs, Addy Osmani's long-form on loop engineering, and recent measurement studies.

Three tiers: figure out if you actually need a loop, learn the five building blocks, then build the smallest one that works without hurting you.

14 steps. 3 tiers. Stop prompting. Start designing.

PART 1 · **The Why & The Test**

01. Loop engineering is replacing yourself *as the prompter*.

For two years, the way you got something out of a coding agent was: write a prompt, share the context, read what came back, write the next prompt. The agent was a tool and you held it the entire time. **That part is ending.**

Victor del Rosal
@victordelrosal

Loop engineering is building a small system that *finds* the work, *hands* it to the agent, *checks* the result, *records* what happened, and *decides* the next move - on its own. You design that system once. The system prompts the agent from then on.

Addy Osmani breaks it into six parts:

Anthropic engineers now merge eight times as much code per day as they did in 2024 - a figure Anthropic itself calls “almost certainly an overstatement of the true productivity gain.”

The number is debated. The mechanism isn’t: **the leverage point moved from typing prompts to designing the loop that prompts**.

02. Run the 4-condition test *before* you build anything.

Loops earn their cost under four conditions. Miss one and the loop costs more than it returns. The honest take from AlphaSignal’s analysis, and the part most X-threads skip:

The four conditions in plain English:

- **The task repeats.** A loop amortizes its setup across many runs. For a one-time job, a good prompt is faster and cheaper. If the work does not recur weekly, you don’t have a loop - you have a script you ran once.
- **Verification is automated.** The loop needs something that can fail the work without you in the room. A test suite, a type checker, a linter, a build. No automated check means you’re back in the chair reading every diff - the exact job the loop was supposed to remove.
- **Your token budget can absorb the waste.** Loops re-read context, retry, explore. That burns tokens whether or not the run ships anything. The technique scales with budget, which is why it reads as obvious to people with effectively free tokens and reckless to people on a metered plan.

Victor del Rosal
@victordelrosal

- **The agent has a senior engineer's tools.** Logs, a reproduction environment, the ability to run the code it writes and see what breaks. Without that, the loop iterates blind.

03. Who wins, who loses. *Loops favor whoever can spend.*

The economics are not universal. The people calling loop engineering obvious tend to have unmetered tokens.

The people for whom it's reckless are usually on a $20 consumer plan trying to run heavy verification loops without hitting limits or a surprise invoice.

Who actually benefits, in practice:

- **Teams with repetitive, machine-checkable work and the budget to run it** - continuous test triage, dependency bumps, lint-and-fix passes, issue-to-PR drafts on a codebase with strong test coverage.
- **Codebases with strong existing test suites.** If a junior engineer could do the task from a checklist and a test suite would catch their mistakes, a loop fits.
- **Async-first teams with multi-agent patterns already in use.** For these teams, routines are the missing orchestration layer.

Who should skip it, today:

- **Solo builders on consumer plans** - the token bill arrives before the productivity gain does.
- **Anyone working on code with no automated verification.** A loop with no real check is the agent agreeing with itself on repeat.
- **Teams whose real constraint is review capacity rather than typing speed.** A loop generates more code; if review was already the bottleneck, it just makes the queue longer.

For one-off tasks, exploratory work, or anything where "done" is a judgment call, **a single well-aimed prompt still wins**. The honest version of this article is: loop engineering is real, and most developers don't need it yet.

04. The 30-second loop check.

The 4-condition test from step 2 is the strategic decision. This is the tactical one - the checklist you run on a specific task before you turn it into a loop.

Miss one box and keep it as a manual prompt.

- **1. The task happens at least weekly.** Less than weekly → setup cost will never amortize.
- **2. A test, type check, build, or linter can reject bad output.** No automated gate → the agent grades its own homework.
- **3. The agent can run the code it changes.** No reproduction environment → iteration is blind.

- **4. The loop has a hard stop.** Token budget, iteration count, or time limit. Without one, the loop runs until someone notices the bill.
- **5. A human reviews before merge, deploy, or dependency changes.** Anything irreversible needs a human approval gate before action.

Good first loops:

- **CI failure triage** - nightly, scan failures, classify causes, draft fix PRs for the easy ones.
- **Dependency bump PRs** - weekly, scan for updates, test compatibility, open PRs.
- **Lint-and-fix passes** - on every PR open event, apply style fixes automatically.
- **Flaky test reproduction** - loop until a theory survives the test.
- **Issue-to-PR drafts** on code with strong tests, where bad output gets rejected by the suite.

Bad first loops - these need a human in the chair:

- Architecture rewrites
- Auth or payments code
- Production deploys
- Vague product work
- Anything where “done” is a judgment call

PART 2 · The 5 Building Blocks

05. Automations: the heartbeat.

Automations are what make a loop an actual loop and not just one run you did once. They fire on a schedule, on an event, or on a trigger condition. They’re the heartbeat - everything else in the loop hangs off them.

What this looks like in the two tools that matter:

- **Codex.** The Automations tab - pick a project, set a prompt, set a cadence, choose local checkout or background worktree. Runs that find something land in a Triage inbox; runs that find nothing archive themselves.
- **Claude Code.** Three primitives that compose into the same shape:

  /loop for session-scoped cadence, Desktop scheduled tasks for restart-survival, Routines for laptop-off cloud runs. Pair with hooks for lifecycle events.

Two primitives inside an automation that separate working loops from expensive ones:

Victor del Rosal
@victordelrosal

- **/loop** re-runs on a cadence. Use it when you want regular checks regardless of state.
- **/goal** keeps going until a condition you wrote is actually true. A separate small model checks completion, so the agent that wrote the code isn't the one grading it.

This is the maker-vs-checker split applied to the stop condition itself.

```
python

> /loop 30m /goal All tests in test/auth pass and lint is clean.
  Scan src/auth for new failures, propose fixes in claude/auth-fixes,
  open draft PR when goal condition holds.

▲ Claude
  CronCreate(*/30 * * * * : auth quality loop)
  Stop condition: tests pass + lint clean (verified by checker)
✓ Scheduled. Will continue past intermediate completions
  until /goal condition is met by independent checker.
```

06. Worktrees: parallel without chaos.

The second you run more than one agent, the files start colliding. Two agents writing the same file is the same headache as two engineers committing to the same lines without talking first.

A git worktree fixes it - a separate working directory on its own branch sharing the same repo history, so one agent's edits literally cannot touch the other's checkout.

GIF

How it shows up in both tools:

Victor del Rosal
@victordelrosal

- **Codex** builds worktree support in - several threads hit the same repo at once without bumping into each other.
- **Claude Code** exposes git worktree directly, a --worktree flag to open a session in its own checkout, and an isolation: worktree setting on subagents so each helper gets a fresh checkout that cleans itself up after.

Worktrees take away the mechanical collision, but you are still the ceiling. Your review bandwidth decides how many parallel agents you can actually run - not the tool.

07. Skills: write project knowledge *once*. *Read on every run.*

A Skill is how you stop re-explaining the same project context every session like a goldfish. Both tools use the same format: a folder with a SKILL.md inside, holding instructions and metadata, plus optional scripts, references, and assets.

Why this matters specifically for loops: a loop without skills re-derives your whole project context from zero every cycle. **With skills, intent compounds.**

The conventions, build steps, “we don’t do it like this because of that one incident” - written once on the outside, read by every run.

```
python

name: ci-triage
description: Classify CI failures by root cause (env, flake, real bug,
  dependency, infra), draft fixes for the easy ones, escalate the rest.
  Trigger whenever a workflow run fails or on the morning triage loop.
---

# CI triage skill

## Classification rules
- env: missing secret, wrong env var, infra not provisioned. # human
- flake: passes on retry without code change. # retry once, then file
- bug: deterministic failure tied to recent commit. # draft fix
- dependency: failure tied to a version bump. # draft rollback
- infra: timeout, OOM, runner issue. # escalate

## Fix patterns
- Auth tests → check src/auth/middleware first
- Database tests → verify migration applied in CI env
- E2E tests → check selectors against the latest UI snapshot

## Never do
- Disable failing tests — always file as escalation instead
- Modify CI config without human approval
- Touch src/payments/ or src/billing/ (in claude/permissions.md)

## State
Update STATE.md after each run: file paths checked, classifications,
PRs opened, items escalated.
```

08. Connectors: the loop touches your real tools. *Via MCP.*

A loop that can only see the filesystem is a tiny loop. **Connectors**, built on the Model Context Protocol (MCP), let the agent read your issue tracker, query a database, hit a staging API, drop a message in Slack.

Victor del Rosal
@victordelrosal

Codex and Claude Code both speak MCP, so the connector you wrote for one usually just works in the other.

This is the difference between an agent that says “here is the fix” and a loop that *opens the PR, links the Linear ticket, and pings the channel once CI is green*.

The connectors are the reason the loop can act inside your actual environment, not just tell you what it would do if it could.

The connectors that pay back fastest for loop work, in order:

- **GitHub** - read repos, create branches, open PRs, comment on issues, react to webhook events. The single biggest day-one win for any code loop.
- **Linear or Jira** - update tickets as the loop progresses, link PRs back to issues, close items automatically when verification passes.
- **Slack** - post triage results, ping humans on escalations, summarize overnight runs in the morning.
- **Sentry / your error tracker** - let the loop investigate live alerts and draft fixes for the high-frequency ones.

09. Sub-agents: keep the maker away from the checker.

The most useful structural thing in a loop, by far, is splitting the agent that writes from the agent that checks.

Osmani’s framing is exact: the model that wrote the code is “way too nice grading its own homework.” A second agent with different instructions and sometimes a different model catches the stuff the first one talked itself into.

This is the **evaluator-optimizer pattern** from Anthropic’s December 2024 engineering post under a new name. One model generates, another critiques, repeat. The vocabulary going viral in 2026 was documented eighteen months ago.

How sub-agents land in both tools:

- **Codex** only spawns subagents when you ask, runs them at the same time, then folds results back into one answer. You define your own agents as TOML files in .codex/agents/ - name, description, instructions, optional model and reasoning effort.

  Your security reviewer can be a strong model on high effort while your explorer is some fast read-only thing.
- **Claude Code** does the same with subagents in .claude/agents/ and agent teams that pass work between them.

  The usual split: one agent explores, one implements, one verifies against the spec.

The reason it matters specifically inside a loop: the loop runs while you are not watching, so a verifier you actually trust is the only reason you can walk away.

Sub-agents burn more tokens since each one does its own model and tool work - spend them where a second opinion is worth paying for.

PART 3 · **Build It Right or Don't Build It**

10. The state file. *The agent forgets. The file does not.*

This is the piece that sounds too dumb to matter and is actually the spine of every working loop. A markdown file, a Linear board, a JSON state - *anything that lives outside the single conversation* and holds what's done and what is next.

Why this matters: agents have short memory by default. What they learn this session is gone tomorrow unless you write it down.

Osmani's rule: the agent forgets, the repo does not. A loop without persistent state restarts every run; a loop with state resumes.

```
json

# Loop state · ci-triage

## Last run
2026-06-09 03:30 UTC · 7 failures classified, 3 fixes drafted, 4 escala

## In progress
- claude/fix-auth-token-refresh — tests passing locally, awaiting CI
- claude/fix-flaky-payment-webhook — retry pattern applied, monitoring

## Completed today
- claude/bump-axios-1.7.4 → merged (CI green, deps loop verified)
- claude/lint-fix-pass-june-9 → merged

## Escalated to humans
- src/billing/refund.ts — tests failing in 3 ways, root cause unclear
- ci/staging-runner — infra timeouts, not a code issue

## Lessons learned (write here, not in chat)
- 2026-06-08: PowerShell hits TLS 1.2 issue on this Windows runner. Use
- 2026-06-07: tests/e2e/checkout requires Stripe webhook secret in env.

## Stop conditions met since last review
- /goal "all tests pass + lint clean" achieved on commit 3a7b8c1 at 02:
```

Two patterns for where the state file lives:

- **Markdown in the repo** - STATE.md at the root or inside .claude/. Version-controlled. Simple. Diff-readable. Best for solo or small

Victor del Rosal
@victordelrosal

team work.

- **External system (Linear, GitHub Issues, a database)** - survives across repos, queryable, supports team-wide visibility. Best for production loops where multiple humans need to see what the loop is doing.

For long-running loops that risk drifting off the goal, pair the state file with **a standing high-level spec** - VISION.md or AGENTS.md - that the agent rereads each run. State tells the agent *where it is*. The spec tells it *where to go*.

11. The minimum viable loop.

If you passed the 4-condition test in step 2, build the smallest loop that works *before* anything fancy. **Four parts, no swarm.**

The four parts, in plain language:

- **One automation.** A scheduled run that fires on a cadence and stops on a clear condition. Use /loop in Claude Code or an automation in Codex. Pair with /goal when you want it to run until a stated condition holds.
- **One skill.** A single SKILL.md that stores the project context the agent would otherwise re-derive from zero every run.
- **One state file.** A markdown file or a Linear board that records what is done and what is next. Tomorrow's run resumes instead of restarting.
- **One gate.** The test, type check, or build that fails bad work automatically. **This is the part that decides whether the loop helps or just spends.**

Order matters: get one manual run reliable first. Turn it into a skill. Wrap it in a loop. Then schedule it. Skipping ahead is how loops fail in production.

The metric that matters is **cost per accepted change** - not tokens spent, not tasks attempted, not loops scheduled. If your accepted-change rate is below 50% you're doing review work the loop saved you from, and the loop is losing.

12. The Ralph Wiggum loop. ***Loops that fail quietly.***

Engineer Geoffrey Huntley documented this failure mode and named it. An agent meant to emit a completion token *only when finished* emits it

Victor del Rosal
@victordelrosal

early, and the loop exits on a half-done job. Without a hard gate, loops fail quietly and keep spending.

The Ralph Wiggum loop is what happens when:

- **No real verifier.** Just a second agent asked to “review,” no objective signal. Two optimists agreeing.
- **Soft completion conditions.** “Done” defined by the agent’s judgment, not by a test, build, or type check.
- **No hard stops.** Loop continues until something external kills it (rate limit, you noticing) rather than until success is verified.

The fix is the gate from step 11 - **something objective that can fail the work**. A test that passes or fails. A build that compiles or doesn’t. A linter that returns zero or non-zero. Not a verifier that has an opinion.

Other measured failure modes worth knowing:

- **Goal drift over long sessions.** Each summarization step is lossy; “don’t do X” constraints disappear at turn 47. Mitigation: a standing VISION.md or AGENTS.md reread each run.
- **Self-preferential bias.** The agent that wrote the code is too nice grading its own homework. Mitigation: a separate verifier subagent with no exposure to the maker’s reasoning.
- **Agentic laziness.** The loop declares “done enough” at partial completion. Mitigation: /goal with an objective stop condition checked by a fresh model.

13. Comprehension debt and cognitive surrender.

This is the failure mode that gets sharper as the loop gets *better*, not worse. Two named risks, both from Osmani’s essay:

- **Comprehension debt.** The faster the loop ships code you didn’t write, the larger the distance between what the repository contains and what you understand. The bill that hurts is not the token bill. *It is the day you have to debug a system no one on the team has read.*
- **Cognitive surrender.** The pull to stop forming an opinion and accept whatever the loop returns. Designing the loop is the cure when you do it with judgment and the accelerant when you do it to avoid thinking. **Same action, opposite result.**

The mitigations are not technical:

- **Read the diffs.** If you don't read what the loop ships, you're renting comprehension debt at compound interest.
- **Spot-check the gate.** Pick a few PRs the loop opened and verify the test that approved them actually catches the failure mode you care about. Gates rot.
- **Block the loop from architecture work.** Keep it on small, machine-checkable changes. The moment you let it touch judgment calls, comprehension debt accelerates.
- **Pair-design loops with a teammate.** A second pair of eyes when designing the loop catches blind spots the loop will exploit forever otherwise.

14. The security tax. *An unattended loop is an unattended attack surface.*

A loop running unattended is also an attack surface running unattended.

The threat model your loop has to defend against:

- **Generated code shipping unreviewed.** The loop opens PRs faster than a human can read them. Without a gate that includes security checks (SAST, dependency audit, secret scanning), insecure code merges automatically.
- **Skills as injection vectors.** A loop that auto-installs skills inherits every prompt injection hiding in their descriptions. Audit skill sources before installing.
- **Credentials in logs.** Debug logging during a long-running loop scatters secrets across logs you don't monitor. Disable verbose logging in production loops; sanitize what does get logged.
- **Permission scope creep.** A loop tested with read-only permissions gets "just one" write permission added for convenience, then never re-audited. *Re-audit permissions every 30 days.*

§ The mistakes that turn loops into money pits

- **Building a loop without running the 4-condition test.** Step 2 exists for a reason. Most developers fail at least one condition.
- **No objective gate.** A second agent asked to "review" without a test, type check, or build is just a second optimist.
- **One agent doing both writing and verifying.** Self-preferential bias. The maker grades its own homework and it's always "A+."
- **No state file.** Tomorrow's run restarts from zero instead of resuming.
- **Vague stop conditions.** "Done when it looks good" never holds. Use a test, a type pass, or a passing build.
- **No token budget cap.** Loops re-read context and retry. Without a cap, ambitious loops burn 5-10× the tokens you expected.
- **Running loops on a consumer plan with heavy verification.** Token bill or rate limit, one of them gets you.
- **Auto-installing community skills.** 520 of 17,022 audited skills leak credentials. Read the source before installing.

Victor del Rosal
@victordelrosal

- **Loops on judgment-call work.** Architecture, auth, payments, vague product decisions. Keep the loop on lint-and-fix, not strategy.
- **Not reading the diffs.** Comprehension debt at compound interest. The day you debug a system no one has read costs more than the tokens ever did.

Conclusion:

The leverage moved. *Your job did too.*

For two years, the leverage in working with coding agents was at the prompt. Better prompts, better context, better one-shot output.

That phase is ending. The agents got good enough that the next leverage point is one floor up: the system that decides what they work on, when, with what gate, and what state survives between runs.

But the honest version of this story is not that everyone should rush to build loops. **Most developers don't need one yet** - not until the task repeats, verification is automated, the budget can absorb the waste, and the agent has senior engineer tools.

Miss one condition and the loop costs more than it returns.

If you pass the test, build small. **One automation. One skill. One state file. One gate.** Get a manual run reliable. Turn it into a skill. Wrap it in a

4:50 PM · Jun 9, 2026 · 310K Views

Cherny's point isn't that the work got easier. *It's that the leverage point moved.* **Build the loop. Stay the engineer.**

Victor del Rosal
@victordelrosal

Victor del Rosal
@victordelrosal

Victor del Rosal
@victordelrosal

Victor del Rosal
@victordelrosal